SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Varun Kumar B*(192525029) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

VarunKumar B

21-7-26

CO1 - Analytical Assessment

1) W = x + y - z + 20 -> lexical Analysis

$id_1 = id_2 + id_3 - id_4 + 20$

↓

Syntax ~~Lexical~~ Analysis

Code generator

↓

$t_1 = id_2 * id_3$

$t_2 = t_1 - id_4$

$t_3 = t_2 + 20$

$id = t_3$

Code optimization

↓

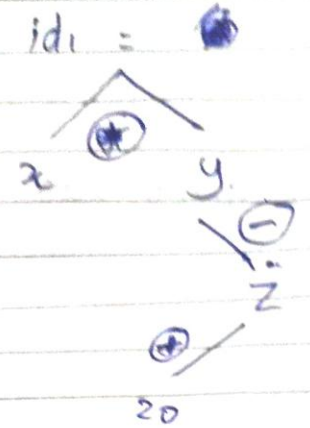
$t_1 = id_2 * id_3$

$id_1 = (t_1 - id_4) + 20.0$

Code generator

↓

```
mov id2, R1
mov id3, R2
mul R2, R1
SUB id4, R1
Add, #20.0, R1
MOV R1, id1
```



- Semantic Analysis.

Parse tree

2.

i) All strings beginning and ending with 1.

$1(0+1)^*1+1$

ii) All strings containing substring 01

$(0+1)^*01(0+1)^*$

iii) All string with exactly one 0

1^*01^*

iv) All string with atleast two 0's

$(0+1)^*1(0+1)^*1(0+1)^*$

v) All string that do not end with 0.

$(0+1)^*1+\varepsilon$

3) i) $(\varepsilon + aa^*)^*$

$(\varepsilon + aa^*)^* = (a^*)^*$

$= a^*$

Any combination of A : { a, aa, aaa ... }

ii) $(a^* + b^*)^*$ 2bb

$(a^* + b^*)^* = (a+b)^*$

$= (a+b)^*$ abb

L = abb

any combination of a & B.

{ ab, aabb, baabb }

iii) $(a^* b^*)^*$ aba

$(a^* + b^*)^* = (a+b)^*$

$= (a+b)^*$ aba

L = aba

any combination of a & B

iv) $(a + ab)^*$

L = { ε, a, aa, ab, aab ... }

v) $(aba)^* + (a^* b^*)^*$

$(a^*b^*)^* = (a|b)^*$

$(aba)^* = \{ \varepsilon, aba, abaaba \ldots \}$

2) i) [a-z A-Z] [a-z A-Z 0-9]*

program

```
%{
#include <stdio.h>
%}

letter [A-Z a-z]
digit [0-9]
ID {letter}{letter}{Digit}*;

%%

if {printf("yy text");}
else {printf(yy text);}
while {printf(yy text);}
{id} {printf(yy text);}

[0-9]+ {printf(yy text);}

" "+ "\n" | "\t" | " " {printf(yy text);}

("||") | {";"} | "D" {print(yy text);}

{.|\n}+;

printf(yy text);}
```

%%

int main () {

printf ("Enter")
yylex()
return 0; }

int yywrap () }
return;
}

5) i) $(0^*1^*)^* = (0+1)^*$

$(0^*1^*)^* = \{\varepsilon, 0, 1, 00, 01, 011, 10 ...\}$

$(0+1)^* = \{\varepsilon, 0, 1, 00, 01, 10, 11 ---\}$

L.H.S = R.H.S

ii) $(0+1)^*(0+1)^* = (0+1)^*$

We know that

$R^* R^* = R^*$

So, $(0+1)^*(0+1)^* = (0+1)^*$

iii) $(0^*)^* = 0^*$

We know that

$(R^*)^* = R^*$

So, $(0^*)^* = 0^*$

L.H.S = R.H.S

iv) $(0+1)^* 0 (0+1)^* + (0+1)^* 1 (0+1)^* = (0+1)^*$

$(0+1)^* 0 + (0+1)^* 1 = (0+1)^*$

$(0+1)^* (1+0)^* = (0+1)^*$

$(0+1)^* = (0+1)^*$

L.H.S = R.H.S

v) $\varepsilon + 0(0)^* = 0^*$

$\varepsilon + R(R)^* = 0^*$

$\varepsilon \neq 1$ L.H.S = R.H.S